```
Student s1 = new Student(); // first object
Student s2 = new Student(); //second object
/*Creating third object*/
s1 = new Student(); // first object becomes eligible for
garbage collection
s2=s1; // second object becomes eligible for garbage
collection
```

In the above code, we have created three objects. As our latest object is the third object, we don't need the first and the second objects. So, we will make them eligible for garbage collection. The third object is now pointed by 's1' so there is no longer any reference pointing to the first object, making it eligible for garbage collection. Similarly, there is no need for the second object as well, so we assign the 's1' reference variable of the third object to 's2' which was earlier pointing to the second object. Now both 's1' and 's2' point to the third object, so the first and second objects become eligible for garbage collection.

Illustration 3-Objects created inside a method

Now let us move on to an illustration of objects inside a method, using three examples.

Example 1

```
Class Student{ }
Class Test{
public static void main(String[] args){
m1();
}
public static void m1(){
Student s1 = new Student();
Student s2 = new Student();
 }
}
```

In the above code, we see that two Student objects are created inside a method; so the moment method *m1()* finishes its execution, the local variables *s1* and *s2* are no longer available. Hence, there is no reference variable at all pointing to either of the two Student objects, in which case, the above two objects become eligible for garbage collection.

Example 2

```
Class Student { }

Class Test {

     public static void main(String[] args){

       Student s =m1();
     }

     public static Student m1(){
       Student s1 = new Student();
       Student s2 = new Student();
       return s1;
     }
}
```

In the above code, we see that method *m1()* returns the *s1* object; so even though *s1* is a local variable, it exists even after the *m1()* method finishes its execution. But *s2* is not returned by the *m1()* method; so once the *m1()* method finishes its execution, *s2* does not exist any more. Hence, in the above code only one object becomes eligible for garbage collection. Let's make a small change in the above code in the *main()* function, as follows:

```
public static void main(String[] args){
m1();
}
```

We know that the *m1()* method return type is Student, but we are not holding the return type while calling the *m1()* method in the *main()* function, as there is no hard and fast rule to compulsorily hold the return type of the method. In such a case, when we don't hold the return type *m1()* method in the *main()* function, then both objects referred by *s1* and *s2* become eligible for garbage collection. In general, the object created inside a method is by default eligible for garbage collection, except in the above exceptional case.

Example 3

```
Class Test {

     static Student s;

     public static void main(String[] args){
       m1();
     }

     public static void m1(){

       s = new Student();
       Student s1 = new Student();
     }
}
```

In the above code, 's' is a static variable, so although the two Student objects are created inside a method, only one object, which is referred by 's1', is eligible for garbage collection. The object that is referred by 's' is not eligible for garbage collection, because 's' is a static variable and not a local one.

Illustration 4-Island of isolation

In the real world, an island is a piece of land surrounded by water. In the case of Java, an island of isolation refers to an